

3D Gaussian Splatting in 100 Lines

Demystifying Real-Time Neural Rendering with Pure PyTorch

Pure PyTorch · No CUDA Required

01

Background & Motivation

Why 3DGS matters

02

Core Concepts

Gaussians, SH, Covariance

03

Math Foundations

Projection, Jacobian, Alpha Compositing

04

Tile-Based Splatting

Culling, Sorting, Tiling

05

Code Walkthrough

~300 lines, zero CUDA

06

Results & Key Learnings

Quality, speed, insights

What is 3D Gaussian Splatting?

3 / 30

A real-time rendering technique that represents 3D scenes as millions of 3D Gaussian blobs — then splats (projects) them onto a 2D screen.

2023

Published at SIGGRAPH by Kerbl et al.

3DGS vs NeRF

Real-time (>30 fps) vs minutes per frame

Representation

Millions of 3D Gaussians with color + opacity

No CUDA needed

Core math is pure linear algebra

100+

fps

Real-time rendering speed
vs NeRF's minutes/frame

**3M+ Gaussians
per scene**

The Problem: Existing Approaches

4 / 30

Before 3DGS, neural scene rendering had painful trade-offs:

NeRF

- Ray-marching through a neural network
- Photorealistic quality
- Minutes per rendered frame
- Not viable for real-time use
- Complex to implement

Meshes

- Fast rasterization (GPU-native)
- Requires manual topology
- Poor for fuzzy/transparent objects
- Hard to reconstruct from photos
- Artists needed

3D Gaussian Splatting

- Learns from photos (like NeRF)
- Real-time rendering (like meshes)
- Handles transparency naturally
- Pure PyTorch implementation
- ~300 lines of code



Core Concepts

Each 3D Gaussian encodes a soft, volumetric blob in space with learned properties:

μ (Mean)

3D center position (x, y, z)

Σ (Covariance)

Shape, size, and orientation of the blob

α (Opacity)

How transparent/solid (0→1)

SH Coefficients

View-dependent color (replaces fixed RGB)



Gaussian
fall-off

The 3D Gaussian is fully defined by a mean vector and a covariance matrix:

$$G(x) = \exp(-\frac{1}{2} \cdot (x - \mu)^T \cdot \Sigma^{-1} \cdot (x - \mu))$$

$\mu \in \mathbb{R}^3$ → center of the Gaussian in 3D space
 $\Sigma \in \mathbb{R}^{3 \times 3}$ → positive semi-definite covariance matrix
 $G(x) \in [0, 1]$ → opacity-like density at position x

Why not just learn Σ directly?

Gradient descent can push Σ into invalid (non-positive definite) territory, breaking the Gaussian. Instead, two unconstrained variables are learned:

Scale vector $s \in \mathbb{R}^3$

Diagonal scale — controls the stretch along each axis

Quaternion $q \in \mathbb{R}^4$

Unit quaternion —

- ✓ Always valid
- ✓ Freely optimizable
- ✓ Decouples shape from rotation

Building a Valid Covariance Matrix

8 / 30

From scale s and quaternion $q \rightarrow$ valid positive-definite Σ :

```
# Step 1: Quaternion → Rotation Matrix
def quat_to_rotmat(q):
    # q = (w, x, y, z) |q| = 1
    R = ... # 3x3 rotation matrix via standard formula

# Step 2: Scale → Diagonal Matrix
S = diag(exp(s)) # exp ensures positive values

# Step 3: Compose the covariance
Σ = R @ S @ S.T @ R.T # Always symmetric + positive-definite
```

Why $\exp(s)$ for scale?

We learn log-scale in unconstrained $\mathbb{R}$. Taking $\exp()$ guarantees positive diagonal values, making the scale matrix valid at all times.

$$\Sigma = R S S^T R^T$$

Always:

- ✓ Symmetric
- ✓ Positive definite
- ✓ Invertible

Spherical Harmonics for View-Dependent Color

9 / 30

Color is not fixed — it changes based on the viewing direction, just like real materials:

```
def evaluate_sh(sh_coeffs, viewing_direction):  
    """  
    sh_coeffs: (N, 3, (degree+1)2) – learned per-Gaussian SH coefficients  
    viewing_direction: unit vector from camera to Gaussian center  
    returns: RGB color (N, 3) for this viewpoint  
    """  
    d = viewing_direction # (x, y, z) normalized  
    # Evaluate basis functions  $Y_l^m(d)$  up to chosen degree  
    color = SH_C0 * sh_coeffs[:, :, 0] # degree-0: constant base color  
    color += SH_C1 * (...) # degree-1: linear view variation  
    color += ... # degree-2,3: specular highlights  
    return sigmoid(color + 0.5) # map to [0, 1]
```

Degree 0

1 coefficient → constant color (no view effect)

Degree 1

4 coefficients → smooth directional color shift

Degree 3

16 coefficients → captures specular highlights



Mathematical Foundations

To project 3D Gaussians onto the screen we need two sets of camera parameters:

Intrinsics K

Describes the internal lens geometry:

- f_x, f_y — focal lengths (pixels)
- c_x, c_y — principal point (image center)
- Controls how 3D $\rightarrow$ 2D mapping scales
- Derived from field of view

$$K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

Extrinsics $[R|t]$

Describes camera pose in world space:

- R — 3x3 rotation matrix
- t — 3D translation vector
- Transforms world coords $\rightarrow$ camera coords
- Computed from SfM/COLMAP output

$$p_{\text{cam}} = R @ p_{\text{world}} + t$$

Projecting 3D Gaussian centers (means) onto the 2D image plane:

```
def project_points(means_3d, viewmat, K):  
    """Project N 3D points into 2D pixel coords"""  
    # 1. World → Camera space  
    p_cam = (viewmat[:3, :3] @ means_3d.T).T + viewmat[:3, 3]  
  
    # 2. Perspective divide  
    x = p_cam[:, 0] / p_cam[:, 2] # x / depth  
    y = p_cam[:, 1] / p_cam[:, 2] # y / depth  
  
    # 3. Apply focal length & principal point  
    u = K[0,0] * x + K[0,2] # pixel column  
    v = K[1,1] * y + K[1,2] # pixel row  
  
    depths = p_cam[:, 2] # needed for depth sorting  
    return torch.stack([u, v], dim=-1), depths
```

Key insight: depth (z) is preserved for later sorting. Gaussians behind the camera are culled after this step.

scale_intrinsics()

If rendering at lower resolution, focal lengths (fx, fy) scale proportionally so FOV stays correct.

3D → 2D Covariance via Jacobian Approximation

13 / 30

Projecting the 3D Gaussian shape onto the screen requires linearizing the perspective transform:

```
# Jacobian of perspective projection (affine approximation)
tx, ty, tz = t_x, t_y, t_z # camera-space mean position

J = (1/tz) * [[fx, 0, -fx*tx/tz],
              [0, fy, -fy*ty/tz]]

# Project 3D covariance  $\Sigma$  to 2D screen covariance  $\Sigma'$ 
W = viewmat[:3, :3] # rotation part of view matrix
 $\Sigma_{2d} = J @ W @ \Sigma_{3d} @ W^T @ J^T$  # 2x2 screen-space covariance
```

The resulting Σ' is a 2x2 matrix that defines the shape of the Gaussian blob on screen:

$\Sigma'[0,0]$

Horizontal spread (pixels²)

$\Sigma'[1,1]$

Vertical spread (pixels²)

$\Sigma'[0,1]$

Shear / rotation on screen

This is the heart of the 'splatting' — the 3D blob becomes a 2D ellipse on screen.

3DGS inherits its pixel color formula from Neural Radiance Fields:

```
# NeRF-style front-to-back alpha compositing
C_pixel =  $\sum_i (\alpha_i \cdot T_i \cdot c_i)$ 
```

where:

```
 $\alpha_i$  = opacity of Gaussian i at this pixel (from 2D Gaussian PDF  $\times$  learned color)
 $T_i$  = transmittance =  $\prod_{j < i} (1 - \alpha_j)$   $\leftarrow$  how much light reaches Gaussian i
 $c_i$  = color of Gaussian i (evaluated from Spherical Harmonics)
```

Transmittance T accumulates as we walk front-to-back:

```
T = 1.0          # start with full transparency
final_color = 0
for each Gaussian i (sorted front to back):
    alpha = pdf_2d(pixel, Gaussian_i) * opacity[i]
    final_color += T * alpha * color[i]
    T *= (1 - alpha)
    if T < 0.0001: break # early termination
```

$T = 1.0$

$\rightarrow$ first Gaussian contributes fully

T decreases each step

$\rightarrow$ later Gaussians

contribute less

$T \approx 0$

$\rightarrow$ pixel is fully opaque

$\rightarrow$ early exit!

Alpha Compositing: The 2D Gaussian PDF

15 / 30

The key quantity is how strongly each Gaussian contributes to a pixel — computed via its 2D PDF:

```
# For pixel p and Gaussian with 2D mean  $\mu'$  and covariance  $\Sigma'$ 
 $\Delta = p - \mu'$  # pixel offset from Gaussian center

power = -0.5 *  $\Delta^T \cdot (\Sigma')^{-1} \cdot \Delta$  # Mahalanobis distance

pdf = exp(power) # ranges [0, 1]; peaks at center

alpha = min(0.99, opacity * pdf) # final per-pixel alpha
```

Center pixel ($\Delta=0$)

power = 0 $\rightarrow$ pdf = 1 $\rightarrow$ maximum contribution

Far pixel (large Δ)

power $\ll$ 0 $\rightarrow$ pdf $\approx$ 0 $\rightarrow$ negligible contribution

This is exactly a 2D Gaussian distribution — that's why the technique is called Gaussian Splatting.

```
# Inverse of 2x2 covariance (fast closed form)
det = cov2d[0]*cov2d[3] - cov2d[1]*cov2d[2]
inv_cov = (1/det) * [[cov2d[3], -cov2d[1]], [-cov2d[2], cov2d[0]]]
```



Tile-Based Splatting

Step 1: Culling

17 / 30

Not every Gaussian contributes to the image — we discard irrelevant ones early:

```
# After projecting all Gaussian means to 2D (u, v):
valid_mask = (
    (depths > 0.01)           # behind camera?
    & (u >= 0) & (u < W)      # off-screen horizontally?
    & (v >= 0) & (v < H)      # off-screen vertically?
)
# Discard everything outside
means_2d = means_2d[valid_mask]
cov_2d   = cov_2d[valid_mask]
colors   = colors[valid_mask]
opacities = opacities[valid_mask]
depths   = depths[valid_mask]
```

Behind Camera

$\text{depth} \leq 0 \rightarrow \text{skip (perspective inversion)}$

Off-Screen

u, v outside image bounds $\rightarrow$ skip entirely

Near-Zero Alpha

$\text{opacity} < \text{threshold} \rightarrow \text{optionally skip}$

Step 2: Depth Sorting

18 / 30

Alpha compositing requires front-to-back ordering — closer Gaussians must come first:

```
# Sort all surviving Gaussians by depth (ascending = front-to-back)
order = torch.argsort(depths)  # indices that sort by depth

means_2d = means_2d[order]
cov_2d   = cov_2d[order]
colors   = colors[order]
opacities = opacities[order]
# depths also reordered — now guaranteed front-to-back
```

Why depth sorting matters:

Without sorting, the transmittance T formula breaks down. A back Gaussian must see 'through' all front Gaussians to know its contribution. If rendered out-of-order, colors blend incorrectly and you get artifacts.

Step 3: Tile Indexing & AABB

19 / 30

The screen is divided into 16×16 pixel tiles. For each Gaussian we compute its Axis-Aligned Bounding Box:

```
TILE_SIZE = 16    # pixels per tile
n_tiles_x = ceil(W / TILE_SIZE)
n_tiles_y = ceil(H / TILE_SIZE)

# Compute 3-sigma radius for each Gaussian's 2D projection
radius = ceil(3 * sqrt(max(cov2d[:, 0, 0], cov2d[:, 1, 1])))

# AABB (Axis-Aligned Bounding Box) in tile space
min_tx = clamp((u - radius) // TILE_SIZE, 0, n_tiles_x)
max_tx = clamp((u + radius) // TILE_SIZE + 1, 0, n_tiles_x)
min_ty = clamp((v - radius) // TILE_SIZE, 0, n_tiles_y)
max_ty = clamp((v + radius) // TILE_SIZE + 1, 0, n_tiles_y)
```

Each Gaussian may overlap multiple tiles — we replicate it once per tile covered:

```
# Build list of (tile_id, gaussian_idx) pairs
for g in range(n_gaussians):
    for tx in range(min_tx[g], max_tx[g]):
        for ty in range(min_ty[g], max_ty[g]):
            tile_id = ty * n_tiles_x + tx
            tile_gaussian_pairs.append((tile_id, g))
```

Step 4: Lexicographic Sort (Tile × Depth)

20 / 30

To enable efficient per-tile processing, we sort all pairs by (tile_id, depth) — a lexicographic sort:

```
tile_ids  = torch.tensor([pair[0] for pair in tile_gaussian_pairs])
gauss_idx = torch.tensor([pair[1] for pair in tile_gaussian_pairs])

# Create combined sort key: tile_id * large_offset + depth_rank
# This sorts primarily by tile, secondarily by depth within tile
sort_keys = tile_ids * 10_000 + depth_rank[gauss_idx]
order = torch.argsort(sort_keys)

tile_ids  = tile_ids[order]
gauss_idx = gauss_idx[order]
# Now: all Gaussians for tile 0 come first (front→back),
#      then tile 1 (front→back), etc.
```

This mirrors the Radix Sort used in the original CUDA paper:

Tile 0, depth 1.2

Tile 0, depth 3.5

Tile 1, depth 0.8

Tile 1, depth 2.1

Tile 2, depth 4.0

*Grouped by tile,
front→back within each tile →
ready for parallel pixel rendering*



Code Walkthrough

Code Structure: ~300 Lines Total

22 / 30

The entire implementation is surprisingly compact:

Helper Functions

~100 lines

evaluate_sh, project_points,
build_covariance, quat_to_rotmat,
scale_intrinsics

Render / Splat Function

~100 lines

Culling, 2D covariance, sorting,
tile indexing, pixel loop,
alpha compositing

Execution Script

~100 lines

Load pretrained params,
camera orbit path,
render loop → frames → GIF

"Stripped of complex engineering for speed, the core rendering mechanism of 3DGS relies entirely on standard linear algebra."

Converts learned SH coefficients → actual RGB color given the current viewing direction:

```
SH_C0 = 0.28209479177387814    # 1/(2√π) – constant term factor

def evaluate_sh(deg, sh, dirs):
    result = SH_C0 * sh[..., 0]          # base color

    if deg > 0:
        x, y, z = dirs[...,0], dirs[...,1], dirs[...,2]
        result += -SH_C1*y*sh[...,1] + SH_C1*z*sh[...,2] - SH_C1*x*sh[...,3]

    if deg > 1:
        xx, yy, zz = x*x, y*y, z*z
        xy, yz, xz = x*y, y*z, x*z
        result += (SH_C2[0]*xy*sh[...,4] + SH_C2[1]*yz*sh[...,5]
                   + SH_C2[2]*(2*zz-xx-yy)*sh[...,6] + ...)

    return torch.clamp(result + 0.5, min=0)    # shift & clamp → [0,1]
```

Degree 0 only: 1 coeff per channel

3 numbers (RGB) → flat, view-independent color. Same from any angle.

Degree 3: 16 coeffs per channel

48 numbers per Gaussian. Captures subtle highlight shifts as camera moves.

Direction = normalize(camera_pos - gaussian_mean) — recalculated every frame.

Helper: build_covariance & quat_to_rotmat

24 / 30

```
def quat_to_rotmat(q):
    """Unit quaternion (w,x,y,z) → 3x3 rotation matrix"""
    w, x, y, z = q[...,0], q[...,1], q[...,2], q[...,3]
    R = torch.stack([
        1-2*(y2+z2), 2*(xy-wz), 2*(xz+wy),
        2*(xy+wz), 1-2*(x2+z2), 2*(yz-wx),
        2*(xz-wy), 2*(yz+wx), 1-2*(x2+y2)
    ], dim=-1).reshape(*q.shape[:-1], 3, 3)
    return R
```

```
def build_covariance(scales, quats):
    """scales ∈ ℝ(N,3), quats ∈ ℝ(N,4) → covariance ∈ ℝ(N,3,3)"""
    R = quat_to_rotmat(quats) # (N, 3, 3)
    S = torch.diag_embed(torch.exp(scales)) # (N, 3, 3) diagonal
    M = R @ S # (N, 3, 3)
    return M @ M.transpose(-1, -2) # Σ = M MT — always PSD
```

⚡ Compute only once after training — camera doesn't change Σ , only the viewing direction for SH changes.

M @ M^T trick

Any matrix M multiplied by its transpose gives a symmetric PSD matrix — a fundamental result from linear algebra.


```
def render(means_3d, covs_3d, sh_coeffs, opacities, K, viewmat, W, H):

    # — 1. Project means —————
    means_2d, depths = project_points(means_3d, viewmat, K)

    # — 2. Cull off-screen Gaussians —————
    mask = (depths > 0.01) & in_image(means_2d, W, H)
    means_2d, depths = means_2d[mask], depths[mask]
    covs_3d, opacities = covs_3d[mask], opacities[mask]
    sh_coeffs = sh_coeffs[mask]

    # — 3. View-dependent colors —————
    view_dir = normalize(means_3d[mask] - camera_pos)
    colors = evaluate_sh(sh_degree, sh_coeffs, view_dir)

    # — 4. 2D covariance via Jacobian —————
    J = compute_jacobian(means_2d, depths, K)          # (N, 2, 3)
    R = viewmat[:3, :3]
    covs_2d = J @ R @ covs_3d @ R.T @ J.T             # (N, 2, 2)
    covs_2d[:, 0, 0] += 0.3                             # low-pass filter
```

Low-pass filter (+0.3)

Adding a small value to $\Sigma'[0,0]$ and $\Sigma'[1,1]$ prevents paper-thin Gaussians that create aliasing artifacts.

Render Function: Pixel Accumulation

26 / 30

```
# — 5. Sort, tile, index —————
order = argsort(depths)
tile_ids, gauss_idxxs = build_tile_index(means_2d[order], covs_2d[order], W, H)

# — 6. Per-tile pixel loop —————
image = torch.zeros(H, W, 3)
for tile_y, tile_x in product(range(n_tiles_y), range(n_tiles_x)):
    tile_mask = (tile_ids == tile_y * n_tiles_x + tile_x)
    g_in_tile = gauss_idxxs[tile_mask]    # Gaussians touching this tile

    for py in range(tile_y*16, min((tile_y+1)*16, H)):
        for px in range(tile_x*16, min((tile_x+1)*16, W)):
            T, C_acc = 1.0, zeros(3)
            for g in g_in_tile:           # front-to-back
                d = [px,py] - means_2d[g]
                pw = -0.5 * d @ inv(covs_2d[g]) @ d
                if pw < -4: continue      # outside 2-sigma
                alpha = min(0.99, opacities[g] * exp(pw))
                C_acc += T * alpha * colors[g]
                T *= (1 - alpha)
                if T < 1e-4: break
            image[py, px] = C_acc
return image
```

Skip pw < -4

Execution Script: Camera Orbit

27 / 30

The main script loads pretrained Gaussian parameters and renders a 360° orbit:

```
# Load pre-trained 3DGS checkpoint (Mip-NeRF 360 indoor scene)
data = load_ply("scene.ply")
means_3d   = data["means"]      # (N, 3)   - 3D positions
sh_coeffs  = data["sh"]         # (N, 3, 16) - color coefficients
scales     = data["scales"]     # (N, 3)   - log scales
quats      = data["quats"]      # (N, 4)   - rotation quaternions
opacities  = data["opacities"]  # (N, 1)   - sigmoid'd opacity

# Pre-compute covariances ONCE (camera-independent)
covs_3d = build_covariance(scales, quats)

# 360° camera orbit
frames = []
for angle in linspace(0, 2*pi, n_frames):
    viewmat = orbit_camera(angle, elevation=20, radius=3.0)
    img = render(means_3d, covs_3d, sh_coeffs, opacities, K, viewmat, W, H)
    frames.append(img)

save_gif(frames, "orbit.gif")
```

Mip-NeRF 360 scenes

Pre-trained checkpoints from Google Research provide photorealistic indoor scenes as test data.

Pre-compute covs_3d

Covariance matrices are camera-independent — compute once for massive speedup across frames.



Results & Key Learnings

Pure PyTorch vs Original CUDA: Quality Comparison

29 / 30

The key finding: visual quality is identical — only speed differs.

⚡ Original CUDA

- C++/CUDA kernels
- 100+ fps real-time
- Complex build system
- GPU-specific code
- Hard to study/modify

🐍 Pure PyTorch

- ~300 lines Python
- Seconds per frame
- pip install torch
- Readable & hackable
- Same PSNR score ✓



Identical PSNR | Same visual quality | Only speed trade-off

Key Learnings & Takeaways

CUDA is optional

The core 3DGS algorithm is pure linear algebra — PyTorch handles it with zero C++.

Depth ordering is critical

Front-to-back sort is mandatory for correct alpha compositing.

Math is simpler than it looks

Perspective projection + Jacobian + alpha compositing = complete renderer.

Tiling enables parallelism

16×16 tiles allow GPU-parallel pixel processing (trivially vectorized).

Pre-compute wisely

Covariance matrices: once. SH colors: every frame. Smart batching matters.

Demystified = improvable

Once you understand the 100 lines, extending 3DGS becomes accessible.